

ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Покрытие кода: тестирование белого ящика

Модуль 2 · Дополнительный блок · Тест-дизайн

Ольга Лихачева · Lektion

4 курс

Факультативная тема

План блока

01 От чёрного ящика к белому: зачем смотреть в код

03 Фрагмент кода: метод `getIssuerId()` сервиса Authorization

05 Уровни покрытия: операторы и ветви

07 MC/DC: покрытие, модифицированное по условию

09 Инструменты измерения покрытия

02 Что такое покрытие кода и как оно измеряется

04 Элементы покрытия и набор тестов TC-01 ... TC-05

06 Уровни покрытия: условия и решения-условия

08 Покрытие путей и его недостижимость

10 Ограничения: сто процентов покрытия не равны корректности

От чёрного ящика к белому

Техники чёрного ящика опираются на требования и описание поведения. **Техники белого ящика** опираются на внутреннее устройство: код и его структуру.

Чёрный ящик: взгляд снаружи

Вход — требования и спецификация.

Вопрос: «что система должна делать?»

Инструмент: КЭ, ГЗ, pairwise, таблицы решений.

Не даёт ответа, все ли ветви кода задействованы.

Белый ящик: взгляд изнутри

Вход — исходный код и его структура.

Вопрос: «какие части кода ещё не выполнялись?»

Инструмент: покрытие операторов, ветвей, условий.

Дополняет чёрный ящик, а не заменяет его.

Почему это не замена: сто процентов покрытия не гарантируют корректности — тесты могут исполнять код, но ничего не проверять.

Что такое покрытие кода

Покрытие кода (code coverage) — доля структурных элементов программы, выполненных при прогоне набора тестов, выраженная в процентах.

Что считаем

Структурный элемент: оператор, ветвь, условие, путь.

На чём считаем

На результате прогона конкретного набора тестов, а не на тесте по отдельности.

Как измеряем

Инструментом: он инструментирован код и фиксирует исполненные элементы.

Формула: **покрытие = выполненные элементы ÷ всего элементов × 100%**.

Уровень покрытия задаётся выбором элемента: что считаем — то и покрываем. Поэтому «уровень покрытия» — это не одно число, а семейство метрик.

Фрагмент кода: метод `getIssuerId()`

Сервис: Authorization · файл: `services/authorization/src/main/java/com/processing/authorization/client/BinLookupClient.java` · метод: `getIssuerId(String pan)`

```
public Optional<String> getIssuerId(String pan) {
    if (pan == null || pan.length() < 6) {
        LOG.warn("Bin lookup skipped: PAN too short");
        return Optional.empty();
    }
    String bin = pan.substring(0, 6);
    try {
        BinLookupResponse response = restClient.get().uri(url + "/api/bin/{bin}", bin)
            .retrieve().body(BinLookupResponse.class);
        String issuerId = response != null ? response.issuerId() : null;
        LOG.debug("Bin lookup OK: bin={} → issuerId={}", bin, issuerId);
        return Optional.ofNullable(issuerId);
    } catch (Exception e) {
        LOG.warn("Bin lookup failed for bin={}: {}", bin, e.getMessage());
        return Optional.empty();
    }
}
```

Условия: A — `pan == null` · B — длина `pan` меньше 6 · C — `response != null`

ИТМО · Тестирование ПО · 4

Решения: guard `A || B` · тернарный оператор по C · `try/catch` → 8 операторов · 5 ветвей · 4 линейных пути

Элементы покрытия: оператор и ветвь

Оператор — минимальная исполняемая инструкция. *ISTQB CTFL v4.0, 4.3*

#	Оператор в коде
1	<code>LOG.warn("PAN too short")</code>
2	<code>return Optional.empty()</code> (в guard)
3	<code>String bin = pan.substring(0, 6)</code>
4	<code>response = ...retrieve().body(...)</code>
5	<code>issuerId = ... ? ... : null</code>
6	<code>LOG.debug("Bin lookup OK...")</code>
7	<code>return Optional.ofNullable(issuerId)</code>
8	тело <code>catch</code> : два оператора

Итого 8 операторов. Условие `if` и вызов `restClient.get()` операторами не являются.

Ветвь — результат вычисления решения. Синоним: решение. *ISTQB CTFL v4.0, 4.3*

#	Ветвь
1	<code>A B</code> → истина → выход
2	<code>A B</code> → ложь → далее
3	<code>try</code> → нормальное завершение
4	тернарный <code>C</code> → ложь
5	<code>catch</code> → исключение

Итого 5 ветвей. Истинная ветвь тернарного оператора относится к ветви 3 и отдельно не считается. Каждому решению отвечают две ветви: `A || B` содержит два условия, но является одним решением.

Элементы покрытия: условие и путь

Условие — атомарное логическое выражение, принимающее значение истина или ложь. *ISTQB Glossary*

Обозн.	Условие в коде	Где
A	<code>pan == null</code>	операнд решения 1
B	<code>pan.length() < 6</code>	операнд решения 1
C	<code>response != null</code>	тернарный оператор

Итого 3 атомарных условия. Выражение `A || B` целиком условием не является: это решение.

Путь — последовательность узлов графа потока управления от входа до выхода. *Куликов, 2.2*

#	Путь
1	вход → <code>A B</code> истина → выход
2	вход → <code>A B</code> ложь → <code>C</code> ложь → выход
3	вход → <code>A B</code> ложь → <code>C</code> истина → выход
4	вход → <code>A B</code> ложь → исключение → выход

Итого 4 пути. Число путей определяется числом решений, а не условий.

Иерархия элементов: 8 операторов → 5 ветвей → 3 условия и 3 решения → 4 пути. Уровни покрытия различаются тем, какое из этих множеств требуется покрыть полностью.

Набор тестов для уровней покрытия

Все уровни рассматриваются на одном фрагменте. Ниже перечислены пять тест-кейсов, из которых собираются наборы.

Кейс	Вход	Что происходит в методе	Зачем нужен
TC-01	валидный PAN, сервис отвечает, issuerId не пуст	операторы 3–7	основной успешный сценарий
TC-02	PAN из 3 цифр	истинная ветвь A B : операторы 1–2	ветвь 1; условие B истинно
TC-03	pan = null	истинная ветвь A B : операторы 1–2	условие A истинно
TC-04	валидный PAN, issuerId = null	ложная ветвь тернарного оператора	ветвь 4; условие C ложно
TC-05	валидный PAN, сервис недоступен	ветвь исключения: оператор 8	ветвь 5; путь 4

TC-03 нужен только для покрытия условий, TC-04 — только для покрытия ветвей.
К покрытию операторов они ничего не добавляют: TC-03 выполняет те же операторы, что TC-02, а TC-04 — те же, что TC-01.

Число тестов по уровням: операторы — 3 (TC-01, 02, 05); ветви — 4 (+ TC-04); условия — 4 (TC-01, 02, 03, 04); решения-условия — 5 (все).

Уровень 1: покрытие операторов

Набор уровня: 3 теста — TC-01, TC-02, TC-05. TC-03 и TC-04 не используются: они выполняют те же операторы.

Покрытие операторов (statement coverage) — доля исполняемых операторов метода, выполненных при прогоне набора тестов.

#	Оператор	TC-01	TC-02	TC-05
1	<code>LOG.warn("PAN too short")</code>	—	✓	—
2	<code>return Optional.empty() — guard</code>	—	✓	—
3	<code>bin = pan.substring(0, 6)</code>	✓	—	✓
4	<code>response = restClient...body()</code>	✓	—	✓
5	<code>issuerId = response ≠ null ? ... : null</code>	✓	—	—
6	<code>LOG.debug("Bin lookup OK...")</code>	✓	—	—
7	<code>return Optional.ofNullable(issuerId)</code>	✓	—	—
8	тело <code>catch</code> : <code>LOG.warn</code> + <code>return empty</code>	—	—	✓

Уровень 2: покрытие ветвей

Набор уровня: 4 теста — TC-01, TC-02, TC-04, TC-05. К набору предыдущего уровня добавлен **TC-04** — единственная непройденная ветвь, ложная ветвь тернарного оператора.

Покрытие ветвей (branch coverage, decision coverage) — доля ветвей метода, выполненных при прогоне набора тестов. Ветвь — каждый возможный результат решения.

Ветвь	Решение	Тест
1	<code>A B</code> → истина (guard сработал)	TC-02, TC-03
2	<code>A B</code> → ложь (валидный PAN)	TC-01, TC-04, TC-05
3	<code>try</code> → нормальное завершение	TC-01, TC-02, TC-04
4	тернарный <code>C</code> → ложь	TC-04
5	<code>catch</code> → исключение	TC-05

Включение: покрытие ветвей включает покрытие операторов, но не наоборот.

Набор — 4 теста: TC-01, TC-02, TC-04, TC-05. Добавляется ровно один — TC-04, закрывающий ветвь 4.

Уровень 3: покрытие условий

Набор уровня: 4 теста — TC-01, TC-02, TC-03, TC-04. Состав **другой**, чем на уровне ветвей: вместо TC-05 взят **TC-03**, дающий истинное значение условия A.

Покрытие условий (condition coverage) — доля атомарных условий, вычисленных в обоих значениях: и истина, и ложь.

Тест	Вход	A: <code>pan == null</code>	B: длина меньше 6	C: <code>response != null</code>
TC-01	валидный PAN	ложь	ложь	истина
TC-02	PAN «123»	ложь	истина	не вычисляется
TC-03	<code>pan = null</code>	истина	не вычисляется	не вычисляется
TC-04	валидный PAN	ложь	ложь	ложь

Короткое замыкание: в TC-03 пан равен null, поэтому правый операнд `A || B` не вычисляется вовсе — условие A засчитывается как истинное.

Непройденная ветвь: все три условия в обоих значениях, но ветвь «исключение» из `try` осталась непройденной. 100% условий не гарантируют 100% ветвей.

Уровень 4: покрытие решений-условий

1

Требование 1: решения

Пять ветвей метода: `guard true` и `false`, `try` — завершение, `catch` — исключение, тернарный — `false`.

2

Требование 2: условия

Три атомарных условия `A`, `B`, `C` — каждое вычислено и в истине, и в лжи.

3

Что даёт вместе

Устраняет пробел предыдущего уровня: ветвь «исключение», непокрытая при 100% условий, теперь обязательна.

4

Набор уровня: 5 тестов

ТС-01 ... ТС-05 — все кейсы. К набору уровня условий (ТС-01, 02, 03, 04) добавлен ТС-05 для ветви исключения.

Уровень 5: MC/DC

Набор уровня: 3 теста на одно решение $A \parallel B$ — **ТС-01** (базовый), **ТС-03** (доказывает влияние A), **ТС-02** (доказывает влияние B). ТС-04 и ТС-05 к этому решению отношения не имеют.

MC/DC — каждое атомарное условие должно **независимо** влиять на результат решения. Покажем на реальном решении $A \parallel B$ нашего метода.

Влияние	Тест	A	B	$A \parallel B$	Вывод
—	ТС-01 (базовый)	ложь	ложь	ложь	базовый случай
A	ТС-03	истина	не вычисляется	истина	A влияет
B	ТС-02	ложь	истина	истина	B влияет

Три теста вместо четырёх: модифицированное покрытие не перебирает все комбинации, а доказывает влияние каждого условия.

Короткое замыкание допустимо: в ТС-03 правый операнд не вычисляется — для MC/DC этого достаточно.

Уровень 6: покрытие путей

Набор уровня: 4 теста — TC-01, TC-02, TC-04, TC-05, по одному на каждый путь. TC-03 не используется: он ведёт по тому же пути, что и TC-02.

Покрытие путей (path coverage) — доля путей через граф потока управления метода, выполненных при прогоне набора тестов.

#	Путь	Тест	Вердикт
1	guard true → выход empty	TC-02, TC-03	линейно независимый
2	guard false → C false → empty	TC-04	базовый
3	guard false → C true → значение	TC-01	линейно независимый
4	guard false → исключение → catch → empty	TC-05	линейно независимый

В методе нет циклов, поэтому полное покрытие путей достижимо — четыре пути. Это редкость: в коде с циклами число путей бесконечно.

Сравнение уровней покрытия

Уровень	Тест-кейсы набора	Тестов	Включение
Операторы	TC-01, TC-02, TC-05	3	—
Ветви (решения)	TC-01, TC-02, TC-04, TC-05	4	включает операторы
Условия	TC-01, TC-02, TC-03, TC-04	4	не включает ветви
Решения-условия	TC-01, TC-02, TC-03, TC-04, TC-05	5	включает оба
MC/DC	TC-01, TC-03, TC-02 (решение A B)	3	включает предыдущий
Пути	TC-01, TC-02, TC-04, TC-05	4	включает решения-условия

TC-03 нужен только для покрытия условий; TC-04 — только для ветвей и путей.

Что осталось за границей покрытия

Все уровни дают 100% на фрагменте — и при этом ни один тест не проверяет *значение* результата.

```
@Test
void smoke() {
    // assertions нет — тест ничего не проверяет
    binLookupClient.getIssuerId(null);
    binLookupClient.getIssuerId("123");
    binLookupClient.getIssuerId("4276010000001111");
}
```

Покрывтие выполнено. Три вызова исполняют все восемь операторов и четыре линейных пути. Инструмент покажет 100%.

Проверок нет. Ни одно значение не сверено с ожидаемым. При неверном `issuerId` прогон останется успешным.

Инструменты измерения покрытия

Как это работает

1. Инструмент инструментует код: вставляет счётчики.
2. Прогоняется набор тестов.
3. Счётчики фиксируют исполненные элементы.
4. Отчёт агрегирует покрытие по уровням.

Инструменты по языкам

Java: JaCoCo, Cobertura.

Go: встроенный `go test -cover`.

Python: coverage.py.

JavaScript / TypeScript: nyc (на основе Istanbul).

```
$ mvn test jacoco:report
$ open target/site/jacoco/index.html
Instruction coverage: 100%
Branch coverage: 100% ← на нашем фрагменте
```

Ограничения: покрытие не равно корректности

Главное ограничение: покрытие измеряет, что код *выполнялся*, а не что результат *проверялся*.

Тест без проверки: три вызова `getIssuerId()` без утверждений дают 100% покрытия при отсутствии проверок результата.

Отсутствующее требование: если бы метод не обрабатывал `pan = null`, в коде не было бы этой ветви. Покрытие покажет 100% — нечего покрывать.

Комбинаторика: 100% ветвей не означают всех комбинаций условий — ложная ветвь тернарного оператора потребовала отдельного `TC-04`.

Данные: покрытие не говорит, были ли использованы граничные значения — PAN ровно из 5 и ровно из 6 цифр.

Приписывать покрытию смысл, которого в нём нет, — распространённая ошибка. Это метрика полноты исполнения, а не метрика качества проверок.

Как использовать покрытие правильно

Полезно как

Индикатор пропущенных участков кода.
Критерий для критичных модулей: покрытие ветвей как норма.
Инструмент поиска мёртвого кода.
Метрика регрессии: покрытие не должно падать.

Бесполезно как

Самоцель: гнаться за ста процентами на любом коде.
Доказательство качества: покрытие не равно корректности.
Замена тест-дизайну: без чёрного ящика набор будет неполным.
Общая метрика по проекту без разбивки по модулям.

Разумный ориентир: покрытие **ветвей** для критичных модулей, покрытие операторов как базовая проверка отсутствия мёртвого кода.

Резюме

Белый ящик опирается на структуру кода; дополняет чёрный ящик, но не заменяет его.

Покрытие кода — доля структурных элементов, исполненных при прогоне набора; измеряется инструментом.

Уровни: операторы → ветви → условия → решения-условия → MC/DC → пути. Строгость растёт, применимость падает.

На одном фрагменте `getIssuerId()` : 8 операторов — 3 теста, 5 ветвей — 4, 4 пути — 4, MC/DC для `A || B` — 3.

Рабочая норма — покрытие ветвей: включает операторы и не требует недостижимых комбинаций.

Главное: покрытие измеряет исполнение, а не корректность. Сто процентов покрытия не означают отсутствия дефектов.

Источники

Источник	Что именно
ISTQB CTFL v4.0	Техники белого ящика: покрытие операторов и ветвей; определения уровней
ISTQB Glossary	Определения: code coverage, statement coverage, branch coverage, decision coverage, condition coverage
Copeland, гл. 4	Структурные техники тестирования, покрытие решений
Куликов, раздел 2.2	Структурные техники, покрытие и его измерение

The background is a dark blue gradient. It features several stylized, glowing green classical buildings with columns, resembling temples or banks. These buildings are interconnected by a network of glowing green lines that resemble circuitry or data flow. There are also several glowing green rectangular shapes, possibly representing tablets or screens, scattered throughout the scene.

Вопросы?